

Métodos Generativos, curso 2026-2027

Guillermo Iglesias, guillermo.iglesias@upm.es

Edgar Talavera Muñoz, e.talavera@upm.es

3 de septiembre de 2026

Escuela Técnica Superior de Ingeniería de Sistemas Informáticos | UPM



Las redes neuronales solo entienden números, no texto.

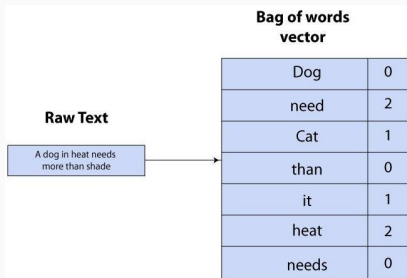
Históricamente, el Natural Language Processing (NLP) ha evolucionado en la forma de transformar palabras en vectores numéricos:

- **Bag of Words (BoW) y TF-IDF:** Representaciones estadísticas clásicas.
- **Word Embeddings (Word2Vec / GloVe):** Representaciones semánticas estáticas.
- **Transformers:** Representaciones contextuales dinámicas.

Bag of Words y TF-IDF

Los primeros enfoques trataban los documentos como "bolsas de palabras", ignorando el **orden y la gramática**.

- **One-Hot Encoding:** Cada palabra del vocabulario es un vector enorme con un '1' en su índice y '0' en el resto.
- **TF-IDF (Term Frequency - Inverse Document Frequency):** Asigna un peso a cada palabra. Penaliza palabras comunes ("a") y da importancia las que definen el texto (e.g. "dog").



[1]

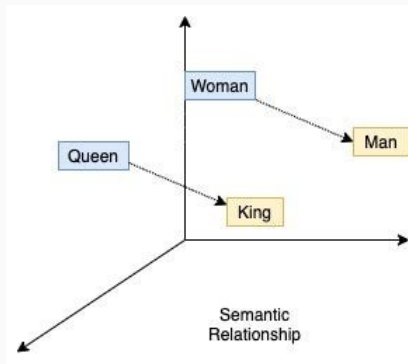
Word Embeddings

Para resolver la falta de significado de TF-IDF, surgieron modelos como **Word2Vec** [2] o **GloVe** [3]. Estos modelos aprenden vectores donde palabras con **significados similares** están **cerca** en el espacio matemático.

- **Aritmética semántica:** Famosamente demostraron que se podía operar con conceptos: *Rey – Hombre + Mujer $\approx$ Reina*.
- **Problema principal (Polisemia):** Estos embeddings son **estáticos**. La palabra "banco" tiene el mismo vector exacto en "banco de dinero" que en "banco de sentarse", impidiendo captar el contexto real.

Word Embeddings

Ahora contamos con un **espacio latente** que representa las palabras siguiendo una **geometría lógica**, que capta la **semántica** de las mismas¹.



[5]

¹Eso no quiere decir que nuestros problemas se hayan acabado, más bien, aquí empiezan otros nuevos... [4]

Motivación y Transformers

El problema secuencial

Para solucionar el problema del contexto, hasta 2017 los problemas secuenciales se trataban (casi) siempre con **Recurrent Neural Network (RNN)** [6].

Estas arquitecturas leen el texto palabra por palabra, actualizando una "memoria". Sin embargo, tienen dos problemas graves:

- **Cuello de botella secuencial:** Su entrenamiento no se puede paralelizar. Para procesar la palabra w , el modelo debe haber procesado la anterior $w - 1$.
- **Olvido catastrófico:** Su memoria es limitada. Empeoran sus resultados drásticamente cuanto más larga es la secuencia.

Transformers

En 2017, investigadores de Google publicaron "*Attention is all you need*" [7] proponiendo las capas de atención:

- **Eficiencia**: Elimina por completo el concepto de recursividad, permitiendo una **paralelización masiva**.
- **Contexto global**: Introduce el concepto de **Atención**, que permite al modelo "ponderar" qué palabras de la secuencia son importantes para entender otra palabra específica, sin importar lo lejos que estén.
- **Contexto**: Resuelve el problema de la polisemia: el vector final de la palabra "banco" cambiará dinámicamente según las palabras que tenga a su alrededor.

Attention is all you need

[PDF] neurips.cc

[A Vaswani](#), [N Shazeer](#), [N Parmar](#)... - Advances in neural ..., 2017 - proceedings.neurips.cc

... to attend to **all** positions in the decoder up to and including that position. **We need** to prevent

... **We** implement this inside of scaled dot-product **attention** by masking out (setting to $-\infty$) ...

☆ Guardar 🔖 Citar Citado por 260796 Artículos relacionados Las 26 versiones 🔗

Tokenización moderna: BPE y Sub-words

Antes de entrar a la arquitectura, hay un último detalle. Los Transformers modernos (como GPT o BERT) no usan palabras completas ni letras individuales, usan **Tokens (Sub-palabras)**.

Algoritmos como **Byte-Pair Encoding (BPE)** [8] dividen las palabras en trozos frecuentes:

- *"Incomprensible"* → *["In", "##comprens", "##ible"]*

Ventajas:

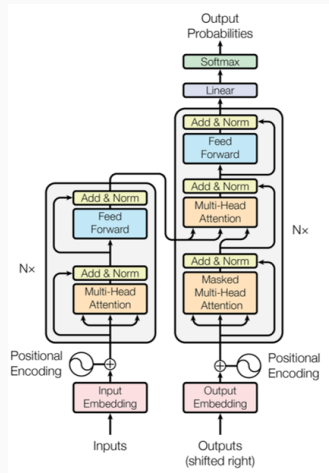
- Evita el problema de las palabras fuera del vocabulario (Out-Of-Vocabulary).
- Permite al modelo entender raíces, prefijos y sufijos, siendo mucho más eficiente.

Arquitectura de los Transformers

Visión General

La arquitectura original del Transformer es capaz de eliminar la recursión. Contiene los siguientes componentes:

- Una arquitectura **Encoder-Decoder**.
- Bloques de atención que deciden para cada palabra qué otras partes son importantes [7].
- **Positional embedding**: Para introducir información temporal o situacional.

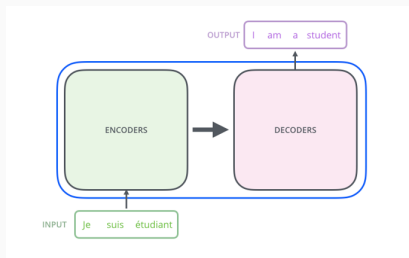


[7]

Visión General

Los Transformers originales nacieron para tareas de *Sequence-to-Sequence* (como la traducción automática). En estos problemas, la entrada y la salida no tienen la misma **longitud** ni el mismo **orden**.

- **Encoder**: Lee toda la frase de entrada y extrae una **representación** con el significado global y el contexto de cada palabra.
- **Decoder**: Actúa como un modelo generativo condicionado. Recibe el "significado" extraído por el Encoder y lo usa para generar la secuencia de destino.



Arquitectura encoder-decoder (fuente).

Generación autorregresiva de secuencias

A la hora de realizar inferencia, el **Decoder** no genera toda la frase de golpe, lo hace de forma **autoregresiva** (paso a paso).

Para generar un nuevo token en el instante t , el modelo calcula la probabilidad basándose en el contexto del Encoder (X) y en **los tokens que ya ha generado previamente**:

$$P(y_t | y_1, y_2, \dots, y_{t-1}, X) \quad (1)$$

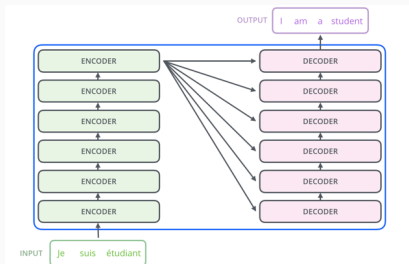
1. Recibe un token de inicio (ej. **[START]**)² y predice la primera palabra.
2. Recibe **[START]** más la palabra anterior, y predice la segunda.
3. El ciclo continúa hasta que el modelo predice un token especial de fin de secuencia (ej. **[END]**).

²Estos son palabras (llamadas tokens) especiales.

Integración de la Atención: El interior del Encoder

Para que el **Encoder** funcione de forma paralela sin dependencias temporales, apila internamente N bloques compuestos por dos capas principales:

- **Self-Attention:** Permite al encoder fijarse en las palabras de la secuencia de entrada que crean el **contexto**.
- **Feed-Forward:** Una red densa clásica que procesa la representación contextualizada de cada vector de forma independiente.



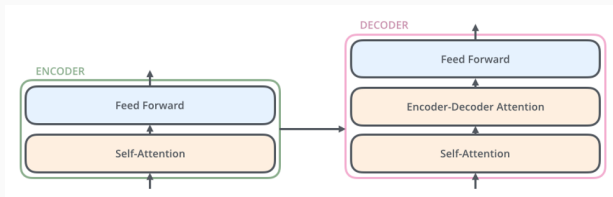
Arquitectura general del transformer (fuente).

Integración de la Atención: El interior del Decoder

El **Decoder** apila N bloques similares. Si le pasamos **toda la frase** real de golpe durante el entrenamiento, el modelo podría usar la siguiente palabra antes de predecirla.

Para evitarlo, la primera sub-capa del Decoder es el **Masked Self-Attention**:

- Se aplica una **máscara** a todas las posiciones futuras ($> t$).
- Al intentar predecir la palabra en la posición t , la **atención del modelo** solo pueda recaer sobre las **posiciones anteriores**.



Arquitectura general del encoder y el decoder (fuente).

Encoder-Decoder Attention (Cross-Attention)

Finalmente, el Decoder incorpora una sub-capa intermedia llamada **Encoder-Decoder Attention**.

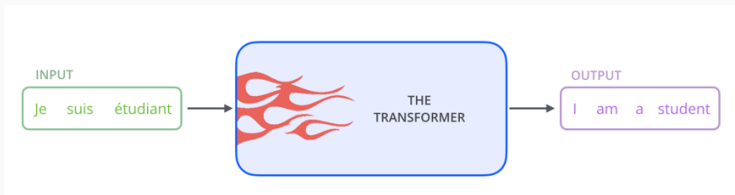
Permite decirle al Decoder qué partes de la secuencia de entrada son **importantes** en ese instante exacto. Sus entradas provienen de flujos distintos:

- **Query (Q):**³ Proviene del propio Decoder (representa la necesidad de información de la palabra que estamos intentando traducir).
- **Key (K) y Value (V):** Proviene de la salida final del Encoder (representan el catálogo de información semántica de la frase original).

³Más adelante entenderemos mejor que es esto de *Query*, *Key* y *Value*.

Arquitectura de los Transformers

- Vamos a ver la arquitectura transformer explicada en el campo del procesamiento del lenguaje natural, ya que es el caso de uso más típico.
- Para el caso de series temporales o imágenes es muy similar, lo único que cambia es la codificación de los datos de entrada.



Ejemplo simplificado de una tarea de traducción con un transformer (fuente).

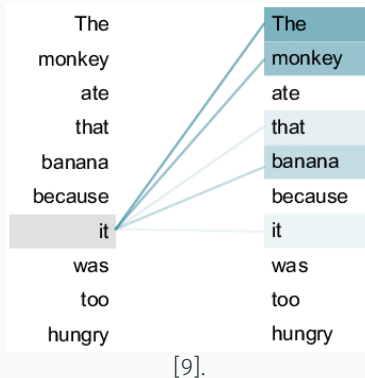
El Mecanismo de Self-Attention

Intuición

En la frase “The *monkey* ate a *bannana* because *it* was hungry”, el modelo no sabe a priori que “it” se refiere a “monkey”.

A través del self-attention:

- El modelo procesa cada palabra (o posición) *mirando* al resto de las posiciones en la secuencia para encontrar *relaciones*.
- Esto le permite *codificar mejor* el elemento actual, integrando el *contexto global*.

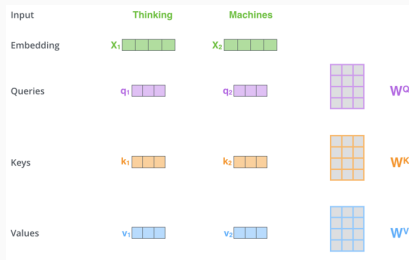


Proyecciones Lineales: Query, Key, Value

A nivel matemático, el proceso comienza proyectando nuestros vectores de entrada a un nuevo espacio dimensional.

Para cada palabra, creamos 3 matrices de pesos entrenables (W^Q, W^K, W^V). Multiplicamos cada **vector de entrada (embedding)** por estas **matrices**⁴ para obtener 3 nuevos vectores:

- **Query (q):** Lo que la palabra "busca" en las demás.
- **Key (k):** Lo que la palabra "ofrece" a las demás.
- **Value (v):** El contenido semántico real que se transferirá.

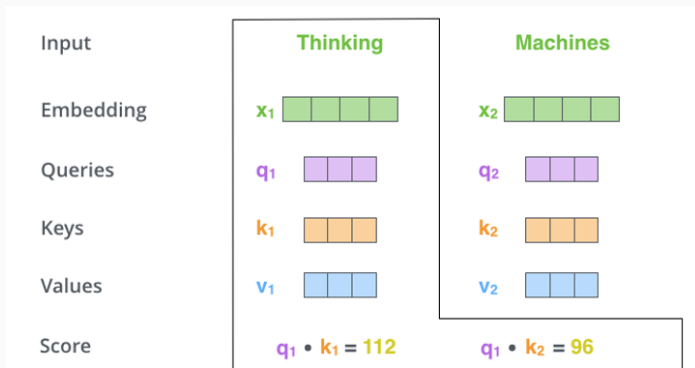


⁴Se escogen de **menor dimensionalidad** que la entrada para reducir el coste (típicamente 64).

Afinidad con Producto Escalar

Una vez tenemos las proyecciones, necesitamos saber cuánto se **relacionan** dos palabras entre sí.

Calculamos la **atención** como el producto escalar de la Query de la palabra actual por la Key de las demás palabras ($q_n \cdot k_m$).



Ejemplo del producto escalar en el mecanismo de self-attention (fuente).

A través del **producto escalar** entre q_n y k_m :

- Geométricamente, el producto escalar mide la **alineación** o similitud entre dos vectores⁵.
- Un resultado alto indica que la palabra m es muy relevante para entender la palabra n .

⁵Cabe recordar que ahora las palabras se representan con vectores, no son puntos en el espacio.

Scaled Dot-Product: Estabilización y Softmax

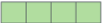
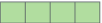
Al calcular productos escalares de vectores en dimensiones altas, los valores resultantes pueden ser inmensos. Esto empuja a la función **softmax** hacia regiones donde el gradiente es casi nulo (saturación).

Para evitarlo y **estabilizar los gradientes** durante el entrenamiento, se realiza un paso crucial:

1. Se divide el resultado entre la raíz cuadrada del número de dimensiones de los vectores key ($\sqrt{d_k}$).
2. Se aplica la función **softmax** para normalizar los valores y hacer que sumen 1.

Scaled Dot-Product: Estabilización y Softmax

Estos valores normalizados (a_n) representan exactamente qué "porcentaje"⁶ de atención debe prestar la palabra actual a cada una de las demás.

Input	Thinking	Machines
Embedding	x_1 	x_2 
Queries	q_1 	q_2 
Keys	k_1 	k_2 
Values	v_1 	v_2 
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12

Ejemplo de la normalización en el mecanismo de self-attention (fuente).

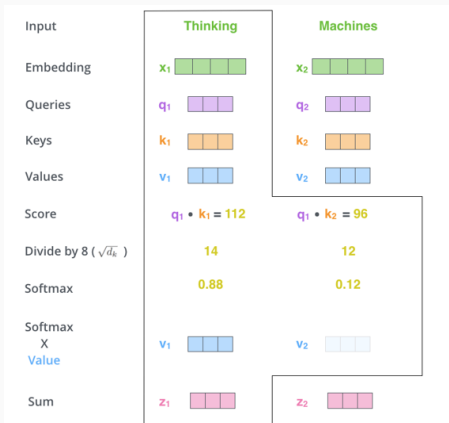
⁶Aunque una softmax técnicamente no proporciona un porcentaje.

El último paso es construir la nueva **representación matemática** de la palabra actual, enriquecida con el contexto.

1. Multiplicamos el valor de **atención normalizado** de cada vector de entrada (a_n) por su vector correspondiente *Value* (v_n). Si la atención es casi 0, ese valor se ahoga pero si es alta, se preserva.
2. **Juntamos todos los resultados** ($a_n \cdot v_n$) para obtener el vector de salida final, z .

Vector de Contexto Final

El vector z es una suma ponderada de la información de todas las palabras de la frase⁷.



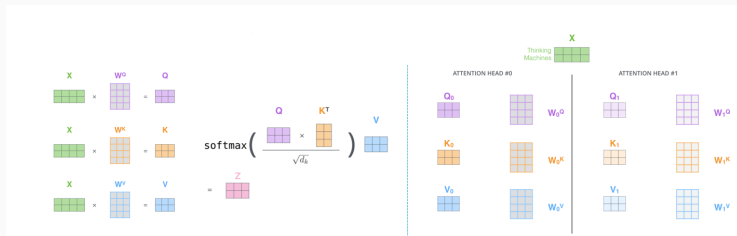
Ejemplo de la salida en el mecanismo de self-attention (fuente).

⁷Y con eso hemos acabado, por fin.

Formulación Matricial: La Ecuación Maestra

En la práctica, para aprovechar el paralelismo de las GPUs, todos estos cálculos paso a paso se agrupan en **operaciones matriciales** para ir más rápido. Toda la mecánica del Self-Attention se resume en esta elegante ecuación:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (2)$$



Ejemplo de la operacion de self-attention (fuente).

Resumen

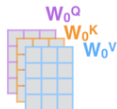
1) This is our input sentence*

Thinking
Machines

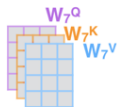
2) We embed each word*



3) Split into 8 heads.
We multiply X or R with weight matrices



...



4) Calculate attention using the resulting $Q/K/V$ matrices



...



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



...



W^O



Z



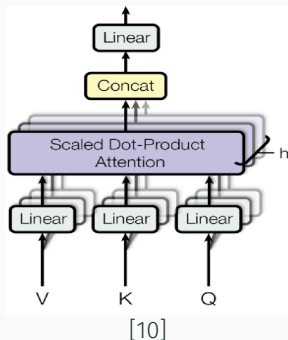
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



Resumen del mecanismo de **self-attention** (fuente).

Multi-head Attention

El mecanismo de **Self-Attention** se combina múltiples veces usando varias matrices de Keys, Queries y Values generando capas de **Multi-head Attention**. Eso mejora las posibilidades de la capa, pudiendo extraer **varios contextos** de la misma secuencia al mismo tiempo⁸.



⁸¿Matando moscas a cañonazos? Quizás, pero así funciona la inteligencia artificial hoy en día.

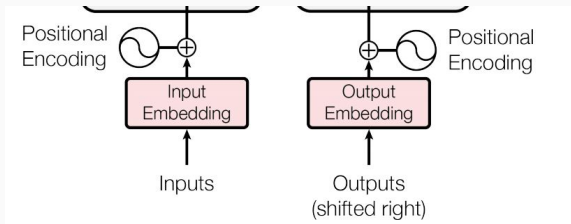
Información temporal con Positional Encoding

El problema de la Invarianza a la Permutación

Al eliminar las redes recurrentes y procesar todos los inputs de **forma paralela**, la arquitectura pierde el orden de la secuencia.

La operación matricial de **Self-Attention** es, matemáticamente, un operador de conjuntos. Esto significa que es **invariante a la permutación**:

- La frase "El perro mordió al hombre" generaría el mismo resultado que "El hombre mordió al perro".
- La red necesita un mecanismo para recuperar la noción del tiempo o la posición.



Positional Encoding

La solución propuesta originalmente [7] es añadir lo que se conoce como **Positional Embedding** (o Encoding).

Modificamos los datos de entrada:

1. Cada palabra se convierte en un vector denso de dimensión d_{model} (ej. 512).
2. Creamos un segundo vector (el *Positional Encoding*) de **mismo tamaño** d_{model} , que contiene un patrón numérico único para esa posición temporal.
3. Se realiza una **suma elemento a elemento** de ambos vectores.

$$\text{Entrada}_{final} = \text{Embedding}_{palabra} + \text{Encoding}_{posicion}$$

Positional Encoding

Estos vectores de posición se generan usando ondas sinusoidales de diferentes frecuencias.

Para una posición temporal pos y una dimensión espacial i , el valor se calcula como:

$$PE_{(pos, 2i)} = \sin \left(\frac{pos}{10000^{2i/d_{model}}} \right) \quad (3)$$

$$PE_{(pos, 2i+1)} = \cos \left(\frac{pos}{10000^{2i/d_{model}}} \right) \quad (4)$$

- Si el índice de la dimensión es par ($2i$), usamos el seno.
- Si el índice de la dimensión es impar ($2i + 1$), usamos el coseno.

¿Por qué senos y cosenos?

Esta formulación matemática parece compleja, pero tiene razones geométricas y de ingeniería muy profundas:

- **Valores limitados:** Los valores oscilan entre $[-1, 1]$, evitando que la suma del vector de posición **sature los valores** del *word embedding*.
- **Frecuencias intercaladas:** Las primeras dimensiones (ondas rápidas) cambian a **cada paso**, mientras que las últimas dimensiones (ondas lentas) cambian **muy despacio**⁹.
- **Relaciones relativas:** Gracias a las propiedades trigonométricas, para cualquier desplazamiento fijo k , PE_{pos+k} se puede representar como una función lineal de PE_{pos} . ¡Esto permite al modelo aprender **distancias relativas** fácilmente!

⁹Funciona como los engranajes de un reloj. Las primeras posiciones del embedding sería el segundero y las últimas la aguja de las horas.

Alternativas del Positional Encoding

Hoy en día, dependiendo de la arquitectura, se usan distintos enfoques:

- **Absolute Learned Embeddings:** Modelos como BERT o GPT-2 no usan senos ni cosenos. Simplemente crean una matriz de pesos entrenables donde la red **aprende** desde cero cuál debe ser el vector para la posición 1, la 2, etc.
- **RoPE (Rotary Positional Embeddings):** Es el estado del arte actual (usado en modelos como LLaMA). En lugar de sumar un vector a la entrada, rota las matrices Q y K en el espacio complejo según su posición relativa justo antes del *Self-Attention*.

Toda esta explicación está basada en la de "The illustrated Transformer", desarrollada por Jay Alammar.

La tenéis disponible en:

<http://jalammar.github.io/illustrated-transformer/>.

Si os habéis quedado con dudas, es **muy recomendable** que os leáis el post y os veáis el video, lo más seguro es que os disipe todas las dudas :-)

Ejemplo de transformer

- Diapositivas de Moodle
- Google Collaboratory
- Deep Learning Book (<https://www.deeplearningbook.org/>)
- <https://www.pyimagesearch.com/blog>
- <https://machinelearningmastery.com/blog>

- [1] Tanesh Balodi.
An optimum approach towards the bag of words with code illustration in python (analytic steps).
<https://www.analyticssteps.com/blogs/an-optimum-approach-towards-the-bag-of-words-with-code-illustration-in-python>, 2019.
[Online; accessed July, 2026].
- [2] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean.
Efficient estimation of word representations in vector space.
arXiv preprint arXiv:1301.3781, 2013.
- [3] Jeffrey Pennington, Richard Socher, and Christopher D Manning.
Glove: Global vectors for word representation.
In Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP), pages 1532–1543, 2014.

- [4] Tolga Bolukbasi, Kai-Wei Chang, James Y Zou, Venkatesh Saligrama, and Adam T Kalai.
Man is to computer programmer as woman is to homemaker? debiasing word embeddings.
Advances in neural information processing systems, 29, 2016.
- [5] Nikhil Birajdar.
Word2vec research paper explained (towards data science).
<https://towardsdatascience.com/word2vec-research-paper-explained-205cb7eecc30/>, 2021.
[Online; accessed July, 2026].
- [6] Robin M Schmidt.
Recurrent neural networks (rnns): A gentle introduction and overview.
arXiv preprint arXiv:1912.05911, 2019.

- [7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin.
Attention is all you need.
In Advances in neural information processing systems, pages 5998–6008, 2017.
- [8] Philip Gage.
A new algorithm for data compression.
The C Users Journal, 12(2):23–38, 1994.
- [9] Huiqiang Xie, Zhijin Qin, Geoffrey Ye Li, and Biing-Hwang Juang.
Deep learning enabled semantic communication systems.
IEEE transactions on signal processing, 69:2663–2675, 2021.

- [10] Fanfei Meng and Chen-Ao Wang.
Sentiment analysis with adaptive multi-head attention in transformer.
arXiv preprint arXiv:2310.14505, 2023.

Cita APA: Iglesias, G. and Talavera, E. (2026, septiembre 3). *Transformers* [Diapositivas de la asignatura Métodos Generativos]. GitHub. ***https:***

//guillermoih.github.io/M-todos-Generativos/Transformers.pdf

Tema de las diapositivas "Metropolis" lincenciado bajo LPPL 1.3c

BibTeX

```
@slides{Transformers_MG_2026,  
  author = {Iglesias, Guillermo and Talavera, Edgar},  
  title = {Transformers [Diapositivas de la asignatura Métodos Generativos]},  
  year = {2026},  
  month = {9},  
  howpublished={\url{https://guillermoih.github.io/M-todos-Generativos/Transformers.pdf}},  
  note = {[Online; accessed 3 de septiembre de 2026]},  
  organization={ETSISI. Universidad Politécnica de Madrid}  
}
```